KATEDRA INFORMATIKY
UNIVERZITA PALACKÉHO V OLOMOUCI
www.inf.upol.cz

Michal Krupka
krupka.inf.upol.cz
michal.krupka@upol.cz
17. listopadu 12, 771 46 Olomouc

Paradigmata programování 2 ⋄ poznámky k přednášce

4. Mutace

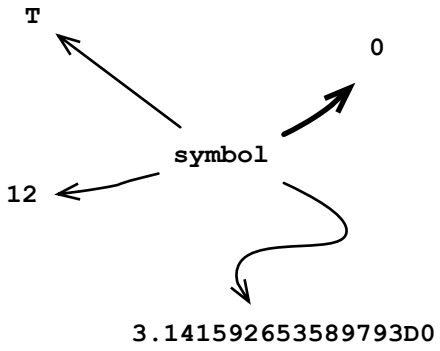
verze z 3. března 2025

Kromě hodnot proměnných lze vedlejším efektem nastavovat i hodnoty složek datových struktur. Na obě tyto možnosti se dnes podrobněji zaměříme. Mutace proměnné i složky datové struktury patří k vedlejším efektům, které označujeme jako *interní*, protože mají vliv na další průběh programu. Pokud máme možnost se mutaci vyhnout, je dobré to udělat.

1 Změna hodnoty vazby symbolu

Nejprve se podíváme, co se přesně děje při nastavování hodnoty symbolu operátorem `setq`.

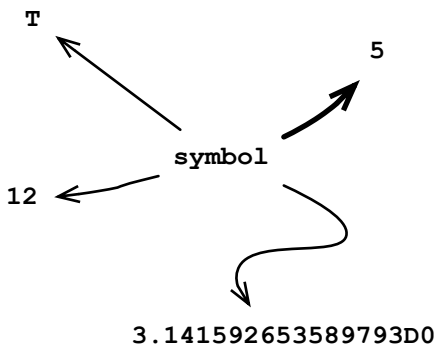
V minulém semestru jsme na jedné přednášce viděli takovýto obrázek:



Šipky znázorňují vazby symbolu `symbol`, tučná šipka je vazba aktuální. Pokud bychom v prostředí s touto vazbou vyhodnotili výraz

```
(setq symbol (+ symbol 5))
```

dopadl by výsledek takto:



Je to proto, že v aktuálním prostředí má symbol nejprve hodnotu `0`, takže výraz `(+ symbol 5)` se vyhodnotí na `5`. Poté se aktuální hodnota symbolu změní. Jeho vyhodnocením (stále ve stejném prostředí) bychom tedy dostali číslo `5`.

Z toho by měl být jasný základní rozdíl mezi speciálním operátorem `setq` a (například) speciálním operátorem `let`. Zatímco operátor `let` **vytváří novou vazbu symbolu**, operátor `setq` **mění hodnotu aktuální vazby symbolu**.

Uvažme tento příklad:

```
(defun two-functions (x)
  (list (lambda () x)
        (lambda (y) (setq x y))))
```

Funkce `two-functions` vrací seznam dvou funkcí, které sdílejí prostředí vzniku. První z těchto funkcí vrací hodnotu proměnné `x` v tomto prostředí, druhá ji nastavuje. Funkci můžeme vyzkoušet:

```
> (bind list1 (two-functions 0))
(#<ANONYMOUS-FUNCTION subfunction of TWO-FUNCTIONS>
#<ANONYMOUS-FUNCTION subfunction of TWO-FUNCTIONS>)

> (funcall (first list1))
0

> (funcall (second list1) 1)
1

> (funcall (first list1))
1
```

Při aplikaci funkce `two-functions` vznikne nové prostředí s vazbou symbolu `x` na hodnotu `0` (pro stručnost v následujících obrázcích neoznačuji, že jde o hodnotové vazby symbolů).

Prostředí funkce `two-functions`

symbol	hodnota
x	0

V tomto prostředí pak funkce vyhodnotí dva λ-výrazy. Jejich výsledkem budou tedy anonymní funkce (lexikální uzávěry), které si budou pamatovat prostředí svého vzniku, tedy prostředí, v němž je symbol `x` navázán na nulu. Tyto funkce budou vráceny ve dvouprvkovém seznamu.

Při zavolání první funkce v seznamu, tedy např. při vyhodnocování výrazu

```
(funcall (first list1))
```

vznikne nové prostředí, jehož předkem je prostředí na předchozím obrázku. Toto prostředí samo neobsahuje žádnou vazbu, protože funkce `(first list1)` nemá žádný parametr. V něm se zjistí hodnota symbolu `x` a vrátí jako výsledek.

Při vyhodnocování výrazu

```
(funcall (second list1) 1)
```

se zavolá druhá funkce seznamu. Ta si také pamatuje prostředí svého vzniku, tedy původní prostředí. Naše tři prostředí budou propojena takto:

Prostředí funkce `two-functions`

symbol	hodnota
x	0

Prostředí první funkce

symbol	hodnota

Prostředí druhé funkce

symbol	hodnota
y	1

V těle druhé anonymní funkce je výraz `(setq x y)`. Ten nejprve vyhodnotí symbol `y` (samozřejmě v aktuálním prostředí, tedy v prostředí označeném na obrázku jako prostředí druhé funkce) a dostane hodnotu `1`. Potom najde aktuální vazbu symbolu `x` a její hodnotu změní.

Po této operaci tedy bude prostředí funkce `two-functions` vypadat takto:

Prostředí funkce `two-functions`

symbol	hodnota
x	1

Proto další volání první funkce, tedy vyhodnocení výrazu

```
(funcall (first list1))
```

bude mít jako výsledek číslo `1`.

Všimněme si důležité (ale nám už dobře známé) skutečnosti. Při aplikaci funkce `two-functions` vznikne nové prostředí, které **nepřestane existovat** ani poté, co aplikace funkce proběhne, a kterému i pak lze měnit hodnoty vazeb.

A druhá důležitá skutečnost: Víme, že při aplikaci funkce (ale také např. při vyhodnocování `let`-výrazu) vzniká **pokaždé nové prostředí**, nezávislé na prostředí předchozím. Následky tohoto faktu se plně projeví při modifikaci hodnot vazeb.

Po dvojí aplikaci funkce tedy budeme mít dvě různá prostředí. V prvním má nyní symbol `x` hodnotu `1`, v druhém může mít jinou. Např. po vyhodnocení

```
> (bind list2 (two-functions 'a))
```

máme dvě prostředí:

Prostředí funkce `two-functions` (první aplikace)

symbol	hodnota
x	1

Prostředí funkce `two-functions` (druhá aplikace)

symbol	hodnota
x	a

Proto můžeme hodnotu vazby symbolu `x` v jednom z nich změnit, aniž by to ovlivnilo druhé prostředí. Obě prostředí můžeme používat současně (přes proměnné `list1` a `list2`).

Příklad. Ukážeme, jak lze reprezentovat páry s mutovatelnými složkami *Car* a *Cdr* pomocí uzávěrů. Pár budeme reprezentovat jako funkci, jejíž prostředí bude obsahovat dvě vazby s informací o složkách. Funkci samotnou bude možné volat s argumentem, který bude určovat, jakou operaci s párem chceme provést. Bude mít i druhý, nepovinný parametr, který bude případně obsahovat požadovanou novou hodnotu jedné ze složek páru.

Funkci bychom tedy mohli definovat takto:

```
(defun my-cons (x y)
  (lambda (what &optional val)
    (cond ((eql what 'car) x)
          ((eql what 'cdr) y)
          ((eql what 'set-car) (setf x val))
          ((eql what 'set-cdr) (setf y val)))))
```

Poznámka: funkci lze zjednodušit použitím makra `case`, se kterým jsme se už setkali minule.

Nyní ještě funkce `my-car`, `my-cdr`, `my-set-car` a `my-set-cdr`, které zjednodušují práci s našimi páry:

```
(defun my-car (c)
  (funcall c 'car))

(defun my-cdr (c)
  (funcall c 'cdr))

(defun my-set-car (c val)
  (funcall c 'set-car val))

(defun my-set-cdr (c val)
  (funcall c 'set-cdr val))
```

Test:

```
> (bind c (my-cons 1 2))
#<ANONYMOUS-FUNCTION subfunction of MY-CONS>

> (my-car c)
1

> (my-cdr c)
2

> (my-set-cdr c 3)
3

> (my-cdr c)
3
```

Abyste dobře porozuměli uvedeným funkcím, můžete si nakreslit příslušná prostředí.

2 Mutace v datových strukturách

Před chvílí jsme si ukázali, jak lze pomocí lexikálních uzávěrů implementovat páry včetně možnosti mutace jejich složek. Nyní se podíváme na mutaci párů používaných v našem Lispu. Jak víme, páry vytváříme konstruktorem `cons` a hodnoty v nich uložené zjišťujeme selektory `car` a `cdr`. K mutaci v párech lze použít funkce (*mutátory*) `set-car` a `set-cdr`. Funkce nastavují příslušnou složku páru a upravený pár vracejí.

```
> (bind pair (cons 1 2))
(1 . 2)
```

```
> (set-car pair 2)
(2 . 2)

> pair
(2 . 2)
```

Jakmile umožníme mutaci složek datové struktury, musíme vést v patrnosti její **totožnost** (**identitu**), tj. rozlišovat, které struktury jsou totožné a které mají jen stejný obsah.

V proměnné `pair` je stále uložen týž pár, jen je změněn obsah jedné jeho složky. To lze demonstrovat tímto testem:

```
> (bind pair2 pair)
(2 . 2)

> (set-cdr pair 1)
(2 . 1)

> pair2
(2 . 1)
```

Tím, že jsme změnili pár v proměnné `pair`, se změnil i pár v proměnné `pair2`. Je to tím, že obě proměnné obsahují tutéž hodnotu:

```
> (eql pair pair2)
T
```

Srovnejte to s tímto pokusem:

```
> (bind pair (cons 1 2))
(1 . 2)

> (bind pair2 (cons 1 2))
(1 . 2)

> (eql pair pair2)
NIL
```

Páry uložené v proměnných `pair` a `pair2` jsou různé, i když na výstupu vypadají stejně. Každý z nich je totiž vytvořen zvláštním voláním konstruktoru `cons`. Mutace jednoho páru tak nemá vliv na druhý:

```
> (set-car pair 3)
3

> pair
(3 . 2)

> pair2
(1 . 2)
```

Poznámka
Páry lze modifikovat, pouze když vznikly za běhu programu použitím konstruktoru `cons`.

Je zakázáno modifikovat páry vzniklé kvotováním ve zdrojovém kódu (jsou chápány jako **literály**):

```
> (bind p '(1 . 2))
(1 . 2)

> (set-car p 2)                                    nesprávně
```

Dále není povoleno modifikovat seznamy uložené v `&rest`-parametrech.

3 Místa

V programovacích jazycích, které nejsou čistě funkcionální, se setkáváme s obecnými **příkazy přiřazení**. Například nastavení hodnoty proměnné v C:

```
a = 10;
```

Na levé straně příkazu přiřazení mohou stát i složitější výrazy než jen proměnné. Efektem přiřazovacího příkazu je pak změna hodnoty složky datové struktury:

```
a[0] = 10;
```

Ve vyšších jazycích než C může programátor nové typy výrazů, které mohou stát na levé straně přiřazovacího příkazu (v C se jim říká **l-hodnoty**), přidávat.

Pomocí **složených příkazů přiřazení** (tak se jim říká v C), pak lze na takových hodnotách vykonávat i některé operace. Takto například lze v C inkrementovat hodnotu složky pole:

```
a[0]++;
```

Podobný nástroj definujeme v našem Lispu. Jeho výhodou bude, že nastavovat hodnoty proměnných i složek různých datových struktur půjde jednotným způsobem:

```
> (bind a 0)
0

> (setf a 1)
1

> a
1

> (bind pair (cons 1 2))
(1 . 2)

> (setf (car pair) 2)
2

> pair
(2 . 2)
```

Pomocí analogií složených operátorů z C budeme moci také hodnoty upravovat (s proměnnými `a` a `pair` z předchozího příkladu):

```
> (incf a)
2

> a
2

> (incf a 8)
10

> a
10

> (incf (car pair) -2)
0

> pair
(0 . 2)
```

V ukázkách jsme použili makra `setf` a `incf`, která (kromě mnoha dalších) za chvíli definujeme. Obě pracují s výrazy, kterým říkáme *místa*.

Použití makra `setf` v jednoduchém tvaru je následující:

```
(setf place expression) => new-value
```

obecně:

```
(setf place1 expression1 ... placen expressionn) => new-value-n
```

Každé místo má dvě **přístupové funkce**: selektor a mutátor

získávají se makrem `place-accessors` Výraz *`place`*, česky **místo**, je výraz, který má dvě vlastnosti:

1. jeho vyhodnocením získáme hodnotu,
2. při použití s makrem `setf`, jak uvedeno výše, se výraz *`expression`* vyhodnotí a makro zařídí, že příště bude hodnotou místa hodnota získaná tímto vyhodnocením.

V obecnějším tvaru makro `setf` umožňuje nastavit hodnoty více míst současně (podívejte se pozorně na výsledek):

```
> (bind a 1)
1

> (bind b 2)
2

> (bind c 3)
3

> (setf a b b c c a)
2

> (list a b c)
(2 3 2)
```

Makro `incf` je příkladem makra pracujícího s místy. Jeho λ-seznam je

```
(place &optional (increment 1))
```

Makro zjistí hodnotu místa (jeho vyhodnocením v aktuálním prostředí), k hodnotě přičte *increment* a výsledek uloží zpět.

Nyní si ukážeme, jak definovat makra pracující s místy, jako je makro `incf`.

Ke každému místu jsou přidruženy dvě funkce: *selektor* a *mutátor*. Selektor je funkce, která po aplikaci na nulový počet argumentů vrátí hodnotu místa, a mutátor funkce, která hodnotu místa nastaví na argument, na který je aplikována, a také jej jako hodnotu vrací.

Základním makrem, které při definici maker pracujících s místy používáme, je makro `place-accessors`. Makro definujeme později, nyní popíšeme, co dělá a jak se využívá. Makro pro dané místo expanduje na výraz, který ve dvouprvkovém seznamu vrací selektor a mutátor místa.

Příklady expanzí makra pro místa, o kterých jsme už mluvili:

```
(place-accessors a)    ⟶  (list (lambda () a)
                                (lambda (val)
                                  (setq a val)))

(place-accessors       ⟶  (let ((p pair))
  (car pair))              (list (lambda () (car p))
                                 (lambda (val)
                                   (set-car p val)
                                   val)))

(place-accessors       ⟶  (let ((p (nth n list)))
  (cdr (nth n list)))      (list (lambda () (cdr p))
                                 (lambda (val)
                                   (set-cdr p val)
                                   val)))
```

Vidíme, že vyhodnocením expandovaných výrazů vždy vznikne dvouprvkový seznam funkcí.

Všimněte si u posledního příkladu, že funkce `nth` se zavolá jen jednou i v případě, že funkce z výsledného seznamu budeme volat vícekrát.

Pomocí makra `place-accessors` nyní můžeme definovat makra pracující s místy. Začneme samotným makrem `setf`.

Makro `setf`

Úkol napsat makro `setf` rozdělíme na dva. Makro `setf-2` bude jeho jednodušší verze pracující pouze s jedním místem. Udělá prostě to, že nechá zavolat mutátor místa. Makro `setf` pak bude makro `setf-2` rekurzivně využívat.

```
(defmacro setf-2 (place value)
  `(funcall (second (place-accessors ,place)) ,value))

(defmacro setf (place value &rest more-places-and-values)
  (if (null more-places-and-values)
      `(setf-2 ,place ,value)
    `(progn (setf-2 ,place ,value)
       (setf ,@more-places-and-values))))
```

Podívejme se, jak bude expandovat jednoduchý výraz s tímto makrem a jeho podvýrazy:

```
(setf (car pair) 10)

         ↓
(setf-2 (car pair) 10)

         ↓

(funcall (second (place-accessors (car pair)))
         10))

         ↓

(funcall (second (let ((p (car pair)))
                   (list (lambda () (cdr p))
                         (lambda (val)
                           (set-cdr p val)
                           val))))
         10))
```

Makro `incf`

Jak má makro pracovat, už víme. Tady je jeho implementace.

```
(defmacro incf (place &optional (increment 1))
  `(bind-list (sel mut) (place-accessors ,place)
     (funcall mut (+ (funcall sel) ,increment))))
```

(Zkuste si sami cvičně expandovat nějaký výraz s tímto makrem.)

Makra `push` a `pop`

Makra jsou určena k práci se seznamy jako se zásobníky. V proměnné, která je argumentem makra, musí být uložen seznam. Makro `push` vytvoří (funkcí `cons`) seznam, jehož prvním prvkem je zadaný prvek a pokračováním seznam v proměnné, a uloží jej zpět do proměnné. Nový seznam také vrátí jako výsledek. Makro `pop` vrátí první prvek seznamu jako výsledek a zbytek seznamu uloží do proměnné:

```
> (bind stack '(1 2 3 4 5))
(1 2 3 4 5)

> (push 0 stack)
(0 1 2 3 4 5)

> stack
(0 1 2 3 4 5)

> (pop stack)
0

> stack
(1 2 3 4 5)

> (pop stack)
1

> stack
(2 3 4 5)
```

Výraz `(push 0 stack)` dělá totéž, co výraz

```
(setf stack (cons 0 stack))
```

výraz `(pop stack)` je ekvivalentní

```
(prog1 (car stack)
  (setf stack (cdr stack)))
```

Makra lze tedy implementovat takto:

```
(defmacro push (elem place)
  `(bind-list (sel mut) (place-accessors ,place)
     (funcall mut (cons 0 (funcall sel)))))

(defmacro pop (place)
  `(bind-list (sel mut) (place-accessors ,place)
     (let ((val (funcall sel)))
       (funcall mut (cdr val))
       (car val))))
```

4 Implementace míst

Nyní se už pusťme do definice míst. Základním a nejdůležitějším krokem je napsat makro `place-accessors`. Pomocí něj na ukázku definujeme dva typy míst: symboly a složené výrazy s operátorem `car`.

Makro musí nejprve rozlišit mezi dvěma typy míst: symboly a složenými výrazy (žádná jiná místa neexistují).

Pro místa, která jsou symboly, makro použije pomocné makro `symbol-accessors`, které expanduje na výraz, jenž vrací v seznamu selektor a mutátor symbolu:

```
(defmacro symbol-accessors (symbol)
  `(list (lambda () ,symbol)
         (lambda (val) (setq ,symbol val))))
```

Dále se podívejme, jak makro `place-accessors` bude pracovat se složenými výrazy. Aby byl složený výraz `(`*`operator`*` ...)` místem, musí existovat jeho tzv. *definiční makro*, což je makro jménem *`operator`*`-accessor`, které pro místo s operátorem *`operator`* expanduje na výraz, který vrací v seznamu selektor a mutátor místa podobně, jak už jsme viděli. λ-seznam definičního makra místa musí odpovídat způsobu, jakým se místo používá.

Například pro místa s operátorem `car` definujeme definiční makro takto:

```
(defmacro car-accessors (pair-expr)
  `(let ((p ,pair-expr))
     (list (lambda () (car p))
           (lambda (val)
             (set-car p val)
             val))))
```

(Mutátor musí vždy vracet nastavovanou hodnotu a funkce `set-car` ji nevrací; proto má tělo mutátoru dva výrazy.)

Makro `place-accessors` bude pro složené výrazy postupovat tak, že zjistí operátor výrazu a pomocí funkce `symbol`, kterou jsem stručně zmínil v minulém textu, vytvoří název definičního makra, tedy symbol *`operator`*`-accessor`.

K vytvoření takového symbolu využije toho, že funkce `stra` přijímá více argumentů, které spojuje dohromady:

```
> (symbol (stra 'car "-ACCESSOR"))
CAR-ACCESSOR
```

A nyní definice makra `place-accessors`:

```
(defmacro place-accessors (place)
  (if (symbolp place)
      `(symbol-accessors ,place)
    `(,(symbol (stra (car place) "-ACCESSORS")) ,@(cdr place))))
```

Jak jsme viděli u místa s operátorem `car`, k definici nového místa je třeba napsat jeho definiční makro.

5 Generátory

Jednotná definice generátoru neexistuje. My jej budeme chápat jako datovou strukturu s jednou hodnotou, která se po každém přístupu může změnit. Generátory budeme implementovat jako funkce bez argumentu, jejichž návratová hodnota se může být pokaždé jiná. Dá se také říci, že funkce postupně různé hodnoty *generuje*. Pro generátor se používá i název iterátor, ale terminologie není jednotná, různí autoři to různě rozlišují, v některých jazycích každý z těchto názvů znamená něco jiného. Generátory (po názvem iterátor) se používají např. v Pythonu.

Takto například můžeme napsat funkci, která vytvoří generátor přirozených čísel:

```
(defun gen-naturals ()
  (let ((n 0))
    (lambda ()
      (setq n (1+ n)))))
```

Ke zjišťování dalších hodnot generátoru budeme používat funkci `next`:

```
(defun next (gen)
  (funcall gen))
```

Test:

```
> (bind gen (gen-naturals))
#<ANONYMOUS-FUNCTION subfunction of FUNCALL>

> (next gen)
1

> (next gen)
2

> (next gen)
```

```
3

> (next gen)
4
```

Tato funkce vrací generátor explicitně zadaných hodnot:

```
(defun gen-values (&rest values)
  (lambda ()
    (prog1 (car values)
      (setf values (cdr values)))))
```

Test:

```
> (bind gen (gen-values 'a 'b 'c))
#<ANONYMOUS-FUNCTION subfunction of GEN-VALUES>

> (next gen)
A

> (next gen)
B

> (next gen)
C
```

Pokus o zjištění další hodnoty vede k chybě (podobně jako v Pythonu):

```
> (next gen)
Error: NIL is not a pair.
```

Vylepšená verze funkce `gen-values` vyvolá po dosažení posledního prvku srozumitelnější chybu:

```
(defun gen-values (&rest values)
  (lambda ()
    (if (null values)
        (error "No values. ")
      (prog1 (car values)
        (setq values (cdr values))))))
```

Nové generátory lze vytvářet z existujících pomocí různých operací. Ukážeme několik příkladů. Všechny generátory, které zde vytvoříme, si sami otestujte.

Funkce `gen+` vytvoří ze dvou zadaných generátorů čísel generátor jejich součtů:

```
(defun gen+ (g1 g2)
  (lambda ()
    (+ (next g1) (next g2))))
```

Funkce `gen-map` vytvoří generátor generující hodnoty vzniklé aplikací zadané funkce na hodnoty generované zadaným generátorem:

```
(defun gen-map (fun gen)
  (lambda ()
    (funcall fun (next gen))))
```

Funkce `gen-filter` vytvoří generátor, který generuje hodnoty generované zadaným generátorem splňující zadaný predikát.

```
(defun gen-filter (pred gen)
  (lambda ()
    (labels ((loop ()
               (let ((res (next gen)))
                 (if (funcall pred res)
                     res
                   (loop)))))
      (loop))))
```

Všechny tři funkce jsou zdánlivě napsány funkcionálně, i když koncová rekurze (v podstatě cyklus) v poslední z nich může budit jisté podezření. Ve skutečnosti mají všechny vedlejší efekt, protože ho má použitá funkce `next`. Generátory do funkcionálního programování nepatří (i když se někdy mylně tvrdí, že ano).

Možné následky předvedeme tímto pokusem. Uložme do proměnné `nat` generátor přirozených čísel

```
> (bind nat (gen-naturals))
#<ANONYMOUS-FUNCTION subfunction of FUNCALL>
```

a zkusme do proměnných `even-nat` a `nat-sq` uložit postupně generátor sudých přirozených čísel a generátor druhých mocnin přirozených čísel:

```
> (bind even-nat (gen-filter #'evenp nat))
#<ANONYMOUS-FUNCTION subfunction of GEN-FILTER>

> (bind nat-sq (gen-map (lambda (x) (* x x))
                        nat))
#<ANONYMOUS-FUNCTION subfunction of GEN-MAP>
```

Náš úmysl ovšem nevyjde podle očekávání, protože generátory pracují s vedlejším efektem:

```
> (next even-nat)
2

> (next even-nat)
4

> (next nat-sq)
25

> (next nat-sq)
36

> (next even-nat)
8
```

6 Kruhové datové struktury

Jak víme, **seznamy** jsou vytvářeny z párů. Konstruktor seznamů (hlavní je funkce `list`) a selektory (například funkce `caar`, `cadr`, atd., `first`, `second` atd., a třeba `nth`) jsou definovány pomocí funkcí `car` a `cdr`.

Ve zdrojovém kódu k této přednášce jsou uvedena definiční makra míst pro většinu těchto operátorů. K **mutaci** seznamů tak můžeme opět používat makro `setf` společně se jmény uvedených funkcí. Například:

```
> (bind list (list 1 4 3))
(1 4 3)

> (setf (second list) 2)
2

> list
(1 2 3)

> (setf (cddr (cdr list)) (list 3 5))
(3 5)

> list
(1 2 3 3 5)

> (setf (nth (+ 1 1) (cdr list)) 4)
```

```
4

> list
(1 2 3 4 5)
```

A teď zajímavá otázka: co dostaneme vyhodnocením těchto výrazů?

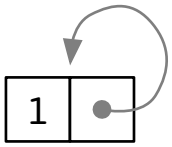
```
> (bind result (list 1))
> (setf (cdr result) result)
```

Na vytvoření datové struktury, kterou vrátí druhý výraz, můžeme napsat i funkci:

```
(defun cl (elem)
  (let ((result (list elem)))
    (setf (cdr result) result)
    result))
```

Funkce vrací seznam, jehož prvním prvkem je zadaný prvek `elem` a zbytkem (*Cdr*) je opět tentýž seznam. Takový seznam jsme v prvním semestru nemohli vyrobit, protože bez použití mutace to nejde.

Strukturu vrácenou vyhodnocením druhého ze shora uvedených výrazů vytvoříme výrazem `(cl 1)`. Krabičkovým znázorněním ji můžeme nakreslit takto:



Lisp bude při jejím tisku do listeneru dávat pozor, zda se nesnaží tisknout pár, který už jednou tiskl. Pokud ano, zavede pro něj značku a místo páru vytiskne ji:

```
> (cl 2)
#1=(2 . #1#)

> (list (cl 3) (cl 4))
(#1=(3 . #1#) #2=(4 . #2#))

> (cl *)
#3=((#1=(3 . #1#) #2=(4 . #2#)) . #3#)
```

Vzhledem k existenci kruhových seznamů musíme vylepšit naše předchozí definice týkající se seznamů. Při té příležitosti a pro úplnost také zavedeme pojem tečkového seznamu. V definici použijeme funkci `nthcdr`.

Nová definice seznamu

Seznam je buď symbol `nil`, nebo pár.

Tečkový seznam je seznam *`list`* takový, že existuje číslo *`n`*, pro které `(nthcdr` *`n list`*`)` není seznam.

Kruhový seznam je seznam *`list`* takový, že existují dvě různá čísla *`m`* a *`n`*, pro která `(nthcdr` *`m list`*`)` ani `(nthcdr` *`n list`*`)` není `nil` a současně `(eql (nthcdr` *`m list`*`) (nthcdr` *`n list`*`))` je *Pravda*, tj. není `nil`.

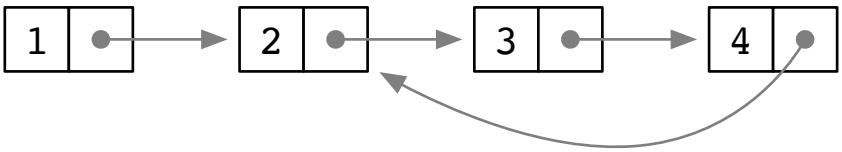
Čistý seznam je seznam, který není ani tečkový, ani kruhový.

Čistému seznamu se v informatice také říká **spojový seznam**. Je to ten druh seznamu, se kterým se v informatice obvykle setkáváme.

Definice čistého seznamu je tady tedy jiná než definice z prvního semestru, kde jsme definovali *čistý seznam délky n* (je s ní ale ekvivalentní).

Obvykle (a je to tak i v jiných předmětech) pokud řekneme *seznam*, myslíme tím pouze čistý seznam. Pokud bychom pracovali se seznamem, který může být i tečkový nebo kruhový, vždy na to upozorníme.

U kruhového seznamu nemusí být ani jedno z čísel *`m`* a *`n`* z definice rovno nule. Toto je příklad takového kruhového seznamu:



Mohl by vzniknout například vyhodnocením výrazu

```
(let ((circle (list 1 2 3 4)))
  (setf (cddddr circle) (cdr circle))
  circle)
```

Mnoho našich funkcí, které pracují se seznamy, předpokládá, že jde o čisté seznamy. Jsou to například funkce `find`, `append` (ta to předpokládá o všech svých argumentech kromě posledního), `length` atd. Z jejich zdrojového kódu je zjevné, že v případě tečkových seznamů může jejich použití vést k chybě a v případě kruhových k zacyklení.

Jak zjistit, zda je daný seznam kruhový? Jeden ze způsobů teď ukážu. Algoritmus využívá techniku **pomalého a rychlého běžce**. Každý z běžců startuje na vstupním seznamu (tj. na jeho prvním páru). Pomalý běžec se pak v každém kroku přesouvá na *Cdr* páru, na kterém zrovna je, rychlý běžec na *Cdr* z *Cdr* páru, na kterém je. Rychlý běžec tedy běží dvakrát rychleji a na konec seznamu dorazí jako první. Pokud je seznam ovšem kruhový, dřív nebo později seznam oběhne a pomalého běžce dožene zezadu.

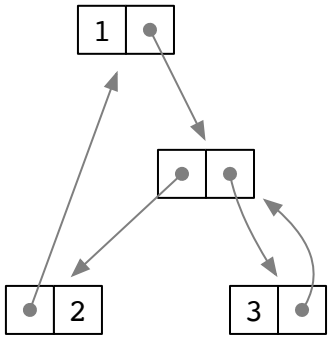
Princip si můžete vyzkoušet na kruhovém seznamu nakresleném na předchozím obrázku. Takto by mohl vypadat predikát, který jej implementuje. Funguje na čisté a kruhové seznamy (na čisté proto, že funkce `cddr` pro prázdný a jednoprvkový seznam vrací `nil`), u tečkových vede k chybě.

```
(defun circlep (list)
  (labels ((cp (slow fast)
             (cond ((null fast) nil)
                   ((eql slow fast) t)
                   (t (cp (cdr slow) (cddr fast))))))
    (cp (cdr list) (cddr list))))
```

Libovolnou hodnotu můžeme chápat jako **strukturu tečkových párů** v tom smyslu, že tečkové páry slouží pouze k propojení hodnot daných atomy. Atom lze chápat jako jednoduchou hodnotu. Pár slouží k uložení dvou struktur tečkových párů (jedné v *car* a druhé v *cdr*), z nichž každá může opět být atomem nebo párem, atd.

Podobnou definici jsme viděli už v minulém semestru. Letošní novinkou je, že struktura tečkových párů může být také kruhová v tom smyslu, že po několikanásobné aplikaci funkcí `car` a `cdr` na pár, který v ní leží, dostaneme týž pár. Struktura také nemusí být kruhová, ale také nemusí být stromová, jak už jsme zjistili v minulém semestru.

Toto je například kruhová struktura:



Chceme-li obecnou strukturu tečkových párů procházet, musíme si pamatovat již navštívené páry. Jinak by mohlo dojít ke dvěma problémům:

1. pokud by struktura nebyla stromová, navštívili bychom některé páry vícekrát;
2. pokud by byla dokonce kruhová, průchod by se zacyklil.

Následující predikát zjišťuje, zda je zadaná struktura tečkových párů stromová:

```
(defun treep (s)
  (let ((visited '()))
    (labels ((tp (node)
               (cond ((find node visited) nil)
                     ((consp node)
                      (push node visited)
                      (and (tp (car node))
                           (tp (cdr node))))
                     (t t))))
      (cs s))))
```